SOCIAL MODULE(MODIFIED)

CSR ACTIVITIES AND EMPLOYEE PARTICIPATION

#models

```
import uuid

from sqlalchemy import Column, String, Text, Integer, Date, ForeignKey

from sqlalchemy.dialects.postgresql import UUID

from database import Base




# ================================
# CSR ACTIVITIES TABLE
# ================================

class CSRActivity(Base):

    __tablename__ = "csr_activities"



    id = Column(
        UUID(as_uuid=True),
        primary_key=True,
        default=uuid.uuid4
    )
```

```
title = Column(
    String(100),
    nullable=False
)


category = Column(
    String(50),
    nullable=False
)


description = Column(
    Text
)


date = Column(
    Date,
    nullable=False
)


status = Column(
    String(20),
    default="ACTIVE"
)
```

```
# ================================
# EMPLOYEE PARTICIPATION TABLE
# ================================

class EmployeeParticipation(Base):

    __tablename__ = "employee_participation"


    id = Column(
        UUID(as_uuid=True),
        primary_key=True,
        default=uuid.uuid4
    )


    employee_id = Column(
        UUID(as_uuid=True),
        ForeignKey("users.id"),
        nullable=False
    )


    activity_id = Column(
        UUID(as_uuid=True),
        ForeignKey("csr_activities.id"),
        nullable=False
    )


    proof = Column(
```

```
        Text,
        nullable=True
    )


    approval_status = Column(
        String(20),
        default="PENDING"
    )


    points = Column(
        Integer,
        default=0
    )


    completion_date = Column(
        Date,
        nullable=True
    )
#schemas
from pydantic import BaseModel
from datetime import date
from uuid import UUID



# Create CSR

class CSRCreate(BaseModel):
```

    title: str

    category: str

    description: str

    date: date

Employee Join

class JoinCSR(BaseModel):

    employee_id: UUID

Upload Proof

class ProofUpload(BaseModel):

    proof: str

Approve Participation

```
class ApprovalRequest(BaseModel):

    status: str



#routes
from fastapi import APIRouter, Depends, HTTPException

from sqlalchemy.orm import Session

from datetime import date

from database import get_db

from .models import (
    CSRActivity,
    EmployeeParticipation
)

from .schemas import (
    CSRCreate,
    JoinCSR,
    ProofUpload,
    ApprovalRequest
)



router = APIRouter(
    prefix="/csr",
```

```
    tags=["CSR"]
)




# ====================================
# CREATE CSR ACTIVITY
# ====================================

@router.post("/")
def create_csr(
    data: CSRCreate,
    db: Session = Depends(get_db)
):

    activity = CSRActivity(

        title=data.title,

        category=data.category,

        description=data.description,

        date=data.date,

        status="ACTIVE"

    )



    db.add(activity)
```

```
    db.commit()

    db.refresh(activity)


    return {

        "message":"CSR Activity Created",

        "activity_id":activity.id

    }




# ====================================
# GET ACTIVE CSR ACTIVITIES
# ====================================

@router.get("/")
def get_csr(

    db:Session=Depends(get_db)

):

    activities = db.query(
        CSRActivity
    ).filter(
```

```
        CSRActivity.status=="ACTIVE"

    ).all()

    return activities


# ====================================
# GET CSR DETAILS
# ====================================

@router.get("/{activity_id}")
def get_csr_details(

    activity_id,

    db:Session=Depends(get_db)

):

    activity=db.query(
        CSRActivity
    ).filter(

        CSRActivity.id==activity_id

    ).first()
```

```
    if not activity:
        raise HTTPException(
            404,
            "Activity not found"
        )

    return activity


# ====================================
# JOIN CSR ACTIVITY
# ====================================

@router.post("/{activity_id}/join")
def join_csr(
    activity_id,
    data:JoinCSR,
    db:Session=Depends(get_db)
```

```
):


    activity=db.query(
        CSRActivity
    ).filter(

        CSRActivity.id==activity_id

    ).first()




    if not activity:

        raise HTTPException(

            404,

            "Activity not found"

        )




    if activity.status!="ACTIVE":

        raise HTTPException(

            400,
```

```
        "Activity closed"

    )




existing=db.query(
    EmployeeParticipation
).filter(

    EmployeeParticipation.employee_id==data.employee_id,

    EmployeeParticipation.activity_id==activity_id

).first()




if existing:

    raise HTTPException(

        400,

        "Already joined"

    )
```

```
participation=EmployeeParticipation(
    employee_id=data.employee_id,
    activity_id=activity_id,
    approval_status="PENDING",
    points=0
)

db.add(participation)
db.commit()
db.refresh(participation)


return {
    "message":"Joined Successfully",
    "participation_id":participation.id
}
```

```
# ====================================
# UPLOAD PROOF
# ====================================

@router.put("/participation/{id}/proof")
def upload_proof(

    id,

    data:ProofUpload,

    db:Session=Depends(get_db)

):



    participation=db.query(
        EmployeeParticipation
    ).filter(

        EmployeeParticipation.id==id

    ).first()



    if not participation:
```

```
        raise HTTPException(
            404,
            "Participation not found"
        )

    participation.proof=data.proof

    db.commit()

    return {
        "message":"Proof uploaded successfully"
    }

# ====================================
# ADMIN APPROVAL
# ====================================
```

```
@router.put("/participation/{id}/approve")
def approve_participation(
    id,
    data:ApprovalRequest,
    db:Session=Depends(get_db)
):

    participation=db.query(
        EmployeeParticipation
    ).filter(
        EmployeeParticipation.id==id
    ).first()


    if not participation:
        raise HTTPException(
            404,
            "Participation not found"
```

```
    )



if data.status=="APPROVED":

    participation.approval_status="APPROVED"

    participation.points=50

    participation.completion_date=date.today()



else:

    participation.approval_status="REJECTED"



db.commit()



return {

    "message":"Participation updated",

    "status":participation.approval_status,
```

```
        "points":participation.points

    }









# ====================================
# EMPLOYEE HISTORY
# ====================================

@router.get("/employee/{employee_id}")
def employee_history(

    employee_id,

    db:Session=Depends(get_db)

):


    records=db.query(

        EmployeeParticipation

    ).filter(

        EmployeeParticipation.employee_id==employee_id
```

).all()

return records